

\$group (aggregation)

Definition

\$group

The `$group` stage separates documents into groups according to a "group key". The output is one document for each unique group key.

A group key is often a field, or group of fields. The group key can also be the result of an expression. Use the `_id` field in the `$group` pipeline stage to set the group key. See below for usage examples.

In the `$group` stage output, the `_id` field is set to the group key for that document.

The output documents can also contain additional fields that are set using accumulator expressions.

NOTE

`$group` does *not* order its output documents.

Compatibility

You can use `$group` for deployments hosted in the following environments:

- MongoDB Atlas: The fully managed service for MongoDB deployments in the cloud
- MongoDB Enterprise: The subscription-based, self-managed version of MongoDB
- MongoDB Community: The source-available, free-to-use, and self-managed version of MongoDB

Syntax

The `$group` stage has the following prototype form:

```
{
  $group:
  {
    _id: <expression>, // Group key
    <field1>: { <accumulator1> : <expression1> },
    ...
  }
}
```



Field	Description
<code>_id</code>	<i>Required.</i> The <code>_id</code> expression specifies the group key. If you specify an <code>_id</code> value of null, or any other constant value, the <code>\$group</code> stage returns a single document that aggregates values across all of the input documents. See the Group by Null example.
<code>field</code>	<i>Optional.</i> Computed using the accumulator operators.

The `_id` and the accumulator operators can accept any valid `expression`. For more information on expressions, see Expression Operators.

Considerations

Accumulator Operator

The `<accumulator>` operator must be one of the following accumulator operators:

Changed in version 5.0.

Name	Description
<code>\$accumulator</code>	Returns the result of a user-defined accumulator function.
<code>\$addToSet</code>	Returns an array of <i>unique</i> expression values for each group. Order of the array elements is undefined. <i>Changed in version 5.0:</i> Available in the <code>\$setWindowFields</code> stage.
<code>\$avg</code>	Returns an average of numerical values. Ignores non-numeric values. <i>Changed in version 5.0:</i> Available in the <code>\$setWindowFields</code> stage.
<code>\$bottom</code>	Returns the bottom element within a group according to the specified sort order. <i>New in version 5.2.</i> Available in the <code>\$group</code> and <code>\$setWindowFields</code> stages.
<code>\$bottomN</code>	Returns an aggregation of the bottom <code>n</code> fields within a group, according to the specified sort order. <i>New in version 5.2.</i> Available in the <code>\$group</code> and <code>\$setWindowFields</code> stages.
<code>\$count</code>	Returns the number of documents in a group. Distinct from the <code>\$count</code> pipeline stage. <i>New in version 5.0:</i> Available in the <code>\$group</code> and <code>\$setWindowFields</code> stages.
<code>\$first</code>	Returns the result of an expression for the first document in a group. <i>Changed in version 5.0:</i> Available in the <code>\$setWindowFields</code> stage.
<code>\$firstN</code>	Returns an aggregation of the first <code>n</code> elements within a group. Only meaningful when documents are in a defined order. Distinct from the <code>\$firstN</code> array operator. <i>New in version 5.2:</i> Available in the <code>\$group</code> , expression and <code>\$setWindowFields</code> stages.
<code>\$last</code>	Returns the result of an expression for the last document in a group. <i>Changed in version 5.0:</i> Available in the <code>\$setWindowFields</code> stage.

Name	Description
<code>\$lastN</code>	Returns an aggregation of the last <code>n</code> elements within a group. Only meaningful when documents are in a defined order. Distinct from the <code>\$lastN</code> array operator. <i>New in version 5.2:</i> Available in the <code>\$group</code>, <code>\$expression</code> and <code>\$setWindowFields</code> stages.
<code>\$max</code>	Returns the highest expression value for each group. <i>Changed in version 5.0:</i> Available in the <code>\$setWindowFields</code> stage.
<code>\$maxN</code>	Returns an aggregation of the <code>n</code> maximum valued elements in a group. Distinct from the <code>\$maxN</code> array operator. <i>New in version 5.2.</i> Available in <code>\$group</code>, <code>\$setWindowFields</code> and as an expression.
<code>\$median</code>	Returns an approximation of the median, the 50th percentile, as a scalar value. <i>New in version 7.0.</i> This operator is available as an accumulator in these stages: <code>\$group</code> <code>\$setWindowFields</code> It is also available as an aggregation expression.
<code>\$mergeObjects</code>	Returns a document created by combining the input documents for each group.
<code>\$min</code>	Returns the lowest expression value for each group. <i>Changed in version 5.0:</i> Available in the <code>\$setWindowFields</code> stage.
<code>\$minN</code>	Returns an aggregation of the <code>n</code> minimum valued elements in a group. Distinct from the <code>\$minN</code> array operator. <i>New in version 5.2.</i> Available in <code>\$group</code>, <code>\$setWindowFields</code> and as an expression.
<code>\$percentile</code>	Returns an array of scalar values that correspond to specified percentile values. <i>New in version 7.0.</i> This operator is available as an accumulator in these stages: <code>\$group</code> <code>\$setWindowFields</code> It is also available as an aggregation expression.
<code>\$push</code>	Returns an array of expression values for documents in each group. <i>Changed in version 5.0:</i> Available in the <code>\$setWindowFields</code> stage.
<code>\$stdDevPop</code>	Returns the population standard deviation of the input values.

Name	Description
	<i>Changed in version 5.0:</i> Available in the <code>\$setWindowFields</code> stage.
<code>\$stdDevSamp</code>	Returns the sample standard deviation of the input values. <i>Changed in version 5.0:</i> Available in the <code>\$setWindowFields</code> stage.
<code>\$sum</code>	Returns a sum of numerical values. Ignores non-numeric values. <i>Changed in version 5.0:</i> Available in the <code>\$setWindowFields</code> stage.
<code>\$top</code>	Returns the top element within a group according to the specified sort order. <i>New in version 5.2.</i> Available in the <code>\$group</code> and <code>\$setWindowFields</code> stages.
<code>\$topN</code>	Returns an aggregation of the top <code>n</code> fields within a group, according to the specified sort order. <i>New in version 5.2.</i> Available in the <code>\$group</code> and <code>\$setWindowFields</code> stages.

`$group` and Memory Restrictions

If the `$group` stage exceeds 100 megabytes of RAM, MongoDB writes data to temporary files. However, if the `allowDiskUse` option is set to `false`, `$group` returns an error. For more information, refer to [Aggregation Pipeline Limits](#).

`$group` Performance Optimizations

This section describes optimizations to improve the performance of `$group`. There are optimizations that you can make manually and optimizations MongoDB makes internally.

Optimization to Return the First or Last Document of Each Group

If a pipeline `sorts` and `groups` by the same field and the `$group` stage only uses the `$first` or `$last` accumulator operator, consider adding an index on the grouped field which matches the sort order. In some cases, the `$group` stage can use the index to quickly find the first or last document of each group.

EXAMPLE

If a collection named `foo` contains an index `{ x: 1, y: 1 }`, the following pipeline can use that index to find the first document of each group:

```
db.foo.aggregate([
  {
    $sort:{ x : 1, y : 1 }
  },
  {
    $group: {
```



```
_id: { x : "$x" },
y: { $first : "$y" }
}
}
])
```

Slot-Based Query Execution Engine

Starting in version 5.2, MongoDB uses the slot-based execution query engine to execute `$group` stages if either:

- `$group` is the first stage in the pipeline.
- All preceding stages in the pipeline can also be executed by the slot-based execution engine.

For more information, see `$group` Optimization.

Examples

Count the Number of Documents in a Collection

In `mongosh`, create a sample collection named `sales` with the following documents:

```
db.sales.insertMany([
  { "_id" : 1, "item" : "abc", "price" : Decimal128("10"), "quantity" :
    Int32("2"), "date" : ISODate("2014-03-01T08:00:00Z") },
  { "_id" : 2, "item" : "jkl", "price" : Decimal128("20"), "quantity" :
    Int32("1"), "date" : ISODate("2014-03-01T09:00:00Z") },
  { "_id" : 3, "item" : "xyz", "price" : Decimal128("5"), "quantity" :
    Int32("10"), "date" : ISODate("2014-03-15T09:00:00Z") },
  { "_id" : 4, "item" : "xyz", "price" : Decimal128("5"), "quantity" :
    Int32("20"), "date" : ISODate("2014-04-04T11:21:39.736Z") },
  { "_id" : 5, "item" : "abc", "price" : Decimal128("10"), "quantity" :
    Int32("10"), "date" : ISODate("2014-04-04T21:23:13.331Z") },
  { "_id" : 6, "item" : "def", "price" : Decimal128("7.5"), "quantity":
    Int32("5") , "date" : ISODate("2015-06-04T05:08:13Z") },
  { "_id" : 7, "item" : "def", "price" : Decimal128("7.5"), "quantity":
    Int32("10") , "date" : ISODate("2015-09-10T08:43:00Z") },
  { "_id" : 8, "item" : "abc", "price" : Decimal128("10"), "quantity" :
    Int32("5") , "date" : ISODate("2016-02-06T20:20:13Z") },
])
```

The following aggregation operation uses the `$group` stage to count the number of documents in the `sales` collection:

```
db.sales.aggregate([
  {
    $group: {
      _id: null,
      count: { $count: { } }
    }
  }
])
```

The operation returns the following result:

```
{ "_id" : null, "count" : 8 }
```

This aggregation operation is equivalent to the following SQL statement:

```
SELECT COUNT(*) AS count FROM sales
```



TIP

See also:

- `$count`
- `$count` (aggregation accumulator)

Retrieve Distinct Values

The following aggregation operation uses the `$group` stage to retrieve the distinct item values from the `sales` collection:

```
db.sales.aggregate( [ { $group : { _id : "$item" } } ] )
```



The operation returns the following result:

```
{ "_id" : "abc" }  
{ "_id" : "jkl" }  
{ "_id" : "def" }  
{ "_id" : "xyz" }
```

Group by Item Having

The following aggregation operation groups documents by the `item` field, calculating the total sale amount per item and returning only the items with total sale amount greater than or equal to 100:

```

db.sales.aggregate(
[
// First Stage
{
$group :
{
_id : "$item",
totalSaleAmount: { $sum: { $multiply: [ "$price", "$quantity" ] } }
},
},
// Second Stage
{
$match: { "totalSaleAmount": { $gte: 100 } }
}
]
)

```

First Stage:

The `$group` stage groups the documents by `item` to retrieve the distinct item values. This stage returns the `totalSaleAmount` for each item.

Second Stage:

The `$match` stage filters the resulting documents to only return items with a `totalSaleAmount` greater than or equal to 100.

The operation returns the following result:

```

{ "_id" : "abc", "totalSaleAmount" : Decimal128("170") }
{ "_id" : "xyz", "totalSaleAmount" : Decimal128("150") }
{ "_id" : "def", "totalSaleAmount" : Decimal128("112.5") }

```

This aggregation operation is equivalent to the following SQL statement:

```

SELECT item,
Sum(( price * quantity )) AS totalSaleAmount
FROM sales
GROUP BY item
HAVING totalSaleAmount >= 100

```

TIP

See also:

`$match`

Calculate Count, Sum, and Average

In `mongosh`, create a sample collection named `sales` with the following documents:

```

db.sales.insertMany([
{ "_id" : 1, "item" : "abc", "price" : Decimal128("10"), "quantity" :
Int32("2"), "date" : ISODate("2014-03-01T08:00:00Z") },
{ "_id" : 2, "item" : "jkl", "price" : Decimal128("20"), "quantity" :
Int32("1"), "date" : ISODate("2014-03-01T09:00:00Z") },

```

```
{ "_id" : 3, "item" : "xyz", "price" : Decimal128("5"), "quantity" :
Int32( "10"), "date" : ISODate("2014-03-15T09:00:00Z") },
{ "_id" : 4, "item" : "xyz", "price" : Decimal128("5"), "quantity" :
Int32("20") , "date" : ISODate("2014-04-04T11:21:39.736Z") },
{ "_id" : 5, "item" : "abc", "price" : Decimal128("10"), "quantity" :
Int32("10") , "date" : ISODate("2014-04-04T21:23:13.331Z") },
{ "_id" : 6, "item" : "def", "price" : Decimal128("7.5"), "quantity":
Int32("5" ) , "date" : ISODate("2015-06-04T05:08:13Z") },
{ "_id" : 7, "item" : "def", "price" : Decimal128("7.5"), "quantity":
Int32("10") , "date" : ISODate("2015-09-10T08:43:00Z") },
{ "_id" : 8, "item" : "abc", "price" : Decimal128("10"), "quantity" :
Int32("5" ) , "date" : ISODate("2016-02-06T20:20:13Z") },
}]
```

Group by Day of the Year

The following pipeline calculates the total sales amount, average sales quantity, and sale count for each day in the year 2014:

```
db.sales.aggregate([
// First Stage
{
  $match : { "date": { $gte: new ISODate("2014-01-01"), $lt: new
ISODate("2015-01-01") } } }
},
// Second Stage
{
  $group : {
    _id : { $dateToString: { format: "%Y-%m-%d", date: "$date" } },
    totalSaleAmount: { $sum: { $multiply: [ "$price", "$quantity" ] } },
    averageQuantity: { $avg: "$quantity" },
    count: { $sum: 1 }
  }
},
// Third Stage
{
  $sort : { totalSaleAmount: -1 }
}
])
```

First Stage:

The `$match` stage filters the documents to only pass documents from the year 2014 to the next stage.

Second Stage:

The `$group` stage groups the documents by date and calculates the total sale amount, average quantity, and total count of the documents in each group.

Third Stage:

The `$sort` stage sorts the results by the total sale amount for each group in descending order.

The operation returns the following results:

```
{
  "_id" : "2014-04-04",
  "totalSaleAmount" : Decimal128("200"),
```

```

"averageQuantity" : 15, "count" : 2
}
{
  "_id" : "2014-03-15",
  "totalSaleAmount" : Decimal128("50"),
  "averageQuantity" : 10, "count" : 1
}
{
  "_id" : "2014-03-01",
  "totalSaleAmount" : Decimal128("40"),
  "averageQuantity" : 1.5, "count" : 2
}

```

This aggregation operation is equivalent to the following SQL statement:

```

SELECT date,
Sum(( price * quantity )) AS totalSaleAmount,
Avg(quantity) AS averageQuantity,
Count(*) AS Count
FROM sales
WHERE date >= '01/01/2014' AND date < '01/01/2015'
GROUP BY date
ORDER BY totalSaleAmount DESC

```

TIP

See also:

- `$match`
- `$sort`
- `db.collection.countDocuments()` which wraps the `$group` aggregation stage with a `$sum` expression.

Group by `null`

The following aggregation operation specifies a group `_id` of `null`, calculating the total sale amount, average quantity, and count of *all* documents in the collection.

```

db.sales.aggregate([
{
  $group : {
    _id : null,
    totalSaleAmount: { $sum: { $multiply: [ "$price", "$quantity" ] } },
    averageQuantity: { $avg: "$quantity" },
    count: { $sum: 1 }
  }
}]

```

The operation returns the following result:

```

{

```

```
"_id" : null,
"totalSaleAmount" : Decimal128("452.5"),
"averageQuantity" : 7.875,
"count" : 8
}
```

This aggregation operation is equivalent to the following SQL statement:

```
SELECT Sum(price * quantity) AS totalSaleAmount,
Avg(quantity) AS averageQuantity,
Count(*) AS Count
FROM sales
```

TIP

See also:

- `$count`
- `db.collection.countDocuments()` which wraps the `$group` aggregation stage with a `$sum` expression.

Pivot Data

In `mongosh`, create a sample collection named `books` with the following documents:

```
db.books.insertMany([
  { "_id" : 8751, "title" : "The Banquet", "author" : "Dante", "copies" : 2 },
  { "_id" : 8752, "title" : "Divine Comedy", "author" : "Dante", "copies" : 1 },
  { "_id" : 8645, "title" : "Eclogues", "author" : "Dante", "copies" : 2 },
  { "_id" : 7000, "title" : "The Odyssey", "author" : "Homer", "copies" : 10 },
  { "_id" : 7020, "title" : "Iliad", "author" : "Homer", "copies" : 10 }
])
```

Group `title` by `author`

The following aggregation operation pivots the data in the `books` collection to have titles grouped by authors.

```
db.books.aggregate([
  { $group : { _id : "$author", books : { $push : "$title" } } }
])
```

The operation returns the following documents:

```
{ "_id" : "Homer", "books" : [ "The Odyssey", "Iliad" ] }
{ "_id" : "Dante", "books" : [ "The Banquet", "Divine Comedy",
"Eclogues" ] }
```

Group Documents by `author`

The following aggregation operation groups documents by `author`:

```
db.books.aggregate([
// First Stage
{
  $group : { _id : "$author", books: { $push: "$$ROOT" } }
},
// Second Stage
{
  $addFields:
  {
    totalCopies : { $sum: "$books.copies" }
  }
}
])
```

First Stage:

`$group` uses the `$$ROOT` system variable to group the entire documents by authors. This stage passes the following documents to the next stage:

```
{ "_id" : "Homer",
  "books" :
  [
    { "_id" : 7000, "title" : "The Odyssey", "author" : "Homer", "copies" : 10 },
    { "_id" : 7020, "title" : "Iliad", "author" : "Homer", "copies" : 10 }
  ]
},
{ "_id" : "Dante",
  "books" :
  [
    { "_id" : 8751, "title" : "The Banquet", "author" : "Dante", "copies" : 2 },
    { "_id" : 8752, "title" : "Divine Comedy", "author" : "Dante", "copies" : 1 },
    { "_id" : 8645, "title" : "Eclogues", "author" : "Dante", "copies" : 2 }
  ]
}
```

Second Stage:

`$addFields` adds a field to the output containing the total copies of books for each author.

NOTE

The resulting documents must not exceed the BSON Document Size limit of 16 megabytes.

The operation returns the following documents:

```
{
```

```
{
  "_id" : "Homer",
  "books" :
  [
    { "_id" : 7000, "title" : "The Odyssey", "author" : "Homer", "copies" : 10 },
    { "_id" : 7020, "title" : "Iliad", "author" : "Homer", "copies" : 10 }
  ],
  "totalCopies" : 20
},
{
  "_id" : "Dante",
  "books" :
  [
    { "_id" : 8751, "title" : "The Banquet", "author" : "Dante", "copies" : 2 },
    { "_id" : 8752, "title" : "Divine Comedy", "author" : "Dante", "copies" : 1 },
    { "_id" : 8645, "title" : "Eclogues", "author" : "Dante", "copies" : 2 }
  ],
  "totalCopies" : 5
}
```

TIP

See also:

`$addFields`

Additional Resources

The Aggregation with the Zip Code Data Set tutorial provides an extensive example of the `$group` operator in a common use case.

